



전자전기컴퓨터설계실험II

실험 전 보고서

순차회로와 카운터의 응용

실험 16-21 · 작성 예시

학과	전자전기컴퓨터공학부
이름 / 학번	이해리 / 2021440107
작성일	2026년 9월 7일
대상	카운터·분주기·스텝모터 구동 논리 피에조·피아노·디지털 시계
검증 환경	Vivado XSim 2026.1 / VS Code VaporView

예시의 범위

기존 예제 소스의 실험 16-21을 대상으로 설계와 사전 시뮬레이션을 정리했다. 실험 번호는 예제 자료의 번호이며, 2026학년도 LAB1 과제 목록을 뜻하지 않는다. 실제 보드 구현·계측은 수업에서 수행할 항목으로 구분했다.

소스 식별

프로젝트: example_project_hdl

기준 커밋: 944f2436e3bd9663a32b4295a8d292b62b327936

실행본: reports/simulation/exp16-exp21

GitHub 주소 / 제출 태그: 이 로컬 작성 예시에는 지정되지 않음. 학생 제출본에는 본인의 저장소 주소·태그·커밋을 기재한다.

1. 목적과 공통 검증 조건

실험 목적

카운터와 상태 레지스터를 사용하여 계수, 주기 생성, 순차 출력 및 시간 표시 회로를 설계한다. 입력 조건을 정하고 출력의 값과 전이 시점을 계산한 뒤 시뮬레이션과 비교한다. 회로의 논리 검증과 보드의 전기적 동작 검증을 구분하는 것이 목표다.

관련 이론

순차회로의 출력과 다음 상태는 현재 입력과 저장된 상태로 결정된다. 동기식 카운터는 클록 에지마다 상태를 갱신하며, 정해진 계수값을 검출하면 주기 신호나 다른 상태의 갱신 조건을 만들 수 있다 [1]. 본 설계는 상승 에지와 active-high 비동기 리셋을 사용한다. 순차 블록의 레지스터 갱신은 nonblocking 할당으로 기술한다.

회로 선택과 소스 구성

top_main.v의 EXPERIMENT를 16~21 중 하나로 정하면 generate 블록이 해당 회로를 선택한다. 사용하지 않는 출력은 상수로 고정된다. tb_top_main.sv는 동일한 입력을 주므로 파형에 나타나는 모든 신호가 각 실험의 유효 출력인 것은 아니다.

실행 방법

원본 src는 유지하고 실험별 실행 폴더에 소스를 복사했다. 각 복사본의 활성 EXPERIMENT 값만 바꾼 뒤 xvlog --sv, xelab, xsim --runall을 실행했다. 결과 top_main.vcd를 VS Code의 VaporView에서 열어 캡처했다. 스크린샷의 시간 단위는 ps이며 $1,000\text{ps} = 1\text{ns}$ 이다.

입력 시나리오

클록은 0에서 시작하여 5 ns마다 반전한다. 리셋은 20 ns에 해제하며 첫 유효 상승 에지는 25 ns이다. 입력은 하강 에지에서 변경한다.

시각(ns)	rst	up	key(hex)
0	1	1	80
20	0	1	80
50,020	0	0	40
100,020	0	0	00
101,020	시뮬레이션 종료		

시간측과 파라미터

실행 클록 주기는 10 ns, 즉 100 MHz다. 반면 음 발생기의 CLK_HZ는 1,000,000, 시계의 값은 1,000으로 유지했다. 따라서 이번 실행은 필요한 클록 개수를 빠르게 검증한다. 예를 들어 시계의 표시 1 초는 시뮬레이션에서 $10\mu\text{s}$ 다. 파형에서 읽은 주파수를 보드의 음높이로 해석하지 않는다.

통과 기준

실험마다 리셋 해제 후 10,100개 상승 에지의 관측 값을 산술·상태 모델과 비교했다. 검증 스크립트 verify_waveforms.py는 VCD의 해당 시각 최종값을 읽어 카운터 값, 분주 비율, 위상 순서, 음 출력과 시계 표시를 검사한다. 6개 실행 모두 종료 시각 101,020 ns와 검사한 출력 조건을 만족했다.

검증 범위

컴파일 성공만으로 기능을 판단하지 않았다. 파형과 기대값을 함께 확인했으며, 기존 테스트벤치가 다루지 않는 입력 조합·긴 시간 경계·실행 중 리셋은 추가 시험 항목으로 남겼다. FPGA 다운로드와 실제 부하 연결은 아직 수행하지 않았다.

2. 실험 16 · 4비트 업/다운 카운터

설계 파일: counter.v / 유효 신호: clk, rst, up, q[3:0]

설계와 예상 결과

리셋 시 $q = 0$ 이며 상승 에지마다 $up=1$ 이면 1 증가, $up=0$ 이면 1 감소한다. 4비트 저장값이므로 $q_{k+1} = (q_k \pm 1) \bmod 16$ 이다. 증가 시 F에서 0, 감소 시 0에서 F로 돌아와야 한다. 리셋 해제 후 25 ns에서 1, 175 ns에서 0을 예상한다.

사전 검증과 파형 해석

그림의 q는 16진수로 표시된다. 초기 1 μ s에서 증가와 순환이 보인다. 50,020 ns의 방향 변경 직전 $q=8$ 이며 다음 상승 에지 50,025 ns에서 7이 된다. 전체 VCD에서 증가·감소 및 경계 순환을 확인했고 종료값은 C다. 방향 전환은 이 초기 확대 그림의 바깥 구간에 있다.

```
example_project_hdl > src > @ counter.v
1 timescale 1ns/1ps
2 module lab5_counter(input wire clk, rst, up, output reg [3:0] q);
3 // up=1: 증가, up=0: 감소. 4비트 범위를 넘으면 순환한다.
4 always @(posedge clk or posedge rst) begin
5     if (rst) q <= 0;
6     else if (up) q <= q + 1'b1;
7     else q <= q - 1'b1;
8 end
9 endmodule
10
```

그림 1: 상승 에지의 증가·감소와 비동기 리셋을 기술한 코드.

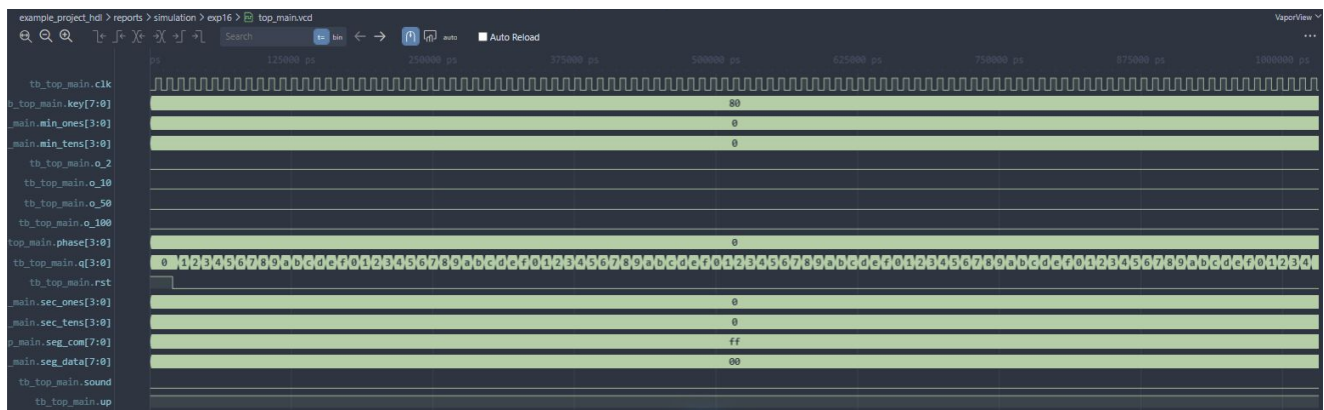


그림 2: VaporView에서 확인한 카운터의 초기 구간. q는 0, 1, 2, ..., F, 0 순으로 변한다.

보드에서 확인할 것

LED 비트 순서를 확인하고 낮은 속도의 계수 조건으로 방향과 순환을 관찰한다. 외부 스위치 입력은 동기화와 디바운스가 필요한지 확인한다.

추가 시험

계수 중 비동기 리셋을 인가하여 즉시 0이 되는지 확인하고, 리셋 해제 후 첫 상승 에지의 동작을 검사한다.

3. 실험 17 · 클록 분주기

설계 파일: clock_divider.v / 유효 출력: o_100, o_50, o_10, o_2

설계와 예상 결과

o_100은 리셋 해제 시 입력 클록을 전달한다. 나머지 출력은 각각 1, 5, 25개 상승 에지마다 반전한다. 출력 한 주기는 반전 두 번이므로 분주비는 각각 2, 10, 50이다. 입력이 100 Hz일 때 출력 이름대로 100, 50, 10, 2 Hz가 된다.

사전 검증과 파형 해석

10 ns 입력에 대한 출력 주기는 10, 20, 100, 500 ns다. VCD의 매 상승 에지에서 분주 출력을 확인했으며 계산한 비율과 일치했다. o_10의 첫 상승은 65 ns, o_2의 첫 상승은 265 ns다. 서로 다른 주기를 보려면 리셋 직후 한 펄스만 비교하지 않고 반복 간격을 읽는다.

```
example_project_hdl > src > @ clock_divider.v > ...
1 timescale 1ns/1ps
2 module lab5_clock_divider(
3     input wire clk, rst,
4     output wire o_100,
5     output reg o_50, o_10, o_2
6 );
7 // 입력 100Hz 기준. 출력은 관찰용이며 다른 회로의 클록으로 쓰지 않는다.
8 reg [2:0] count_10;
9 reg [4:0] count_2;
10 assign o_100 = rst ? 1'b0 : clk;
11 always @(posedge clk or posedge rst) begin
12     if (rst) begin
13         o_50 <= 0; o_10 <= 0; o_2 <= 0;
14         count_10 <= 0; count_2 <= 0;
15     end else begin
16         o_50 <= ~o_50;
17         if (count_10 == 4) begin
18             count_10 <= 0;
19             o_10 <= ~o_10;
20         end else count_10 <= count_10 + 1'b1;
21         if (count_2 == 24) begin
22             count_2 <= 0;
23             o_2 <= ~o_2;
24         end else count_2 <= count_2 + 1'b1;
25     end
26 end
27 endmodule
28
```

그림 3: 계수 종료값 4와 24에서 출력을 반전하고 카운터를 0으로 되돌린다.

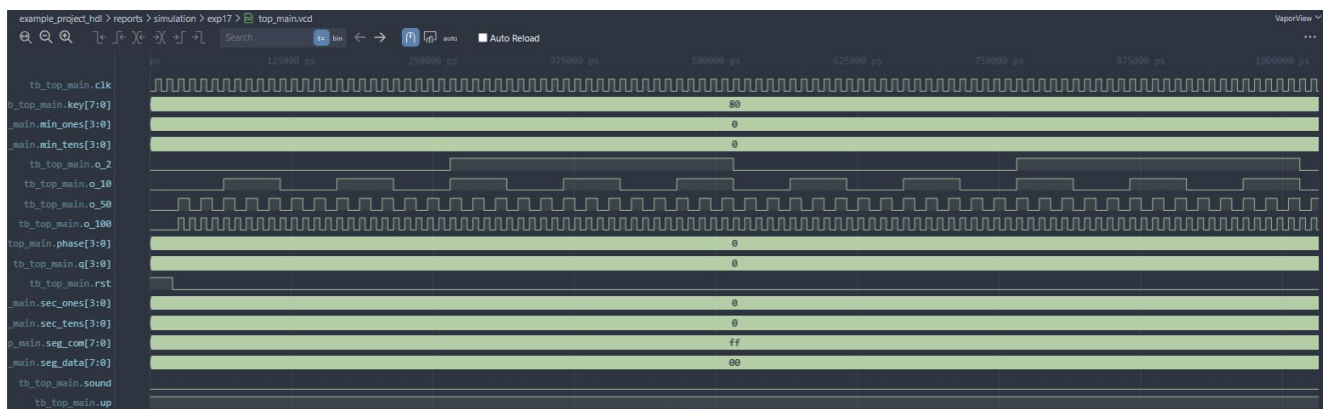


그림 4: 입력 대비 1, 2, 10, 50배인 출력 주기. 이번 시간축의 출력은 Hz 이름과 다르다.

보드 검증 계획

입력의 실제 주파수를 먼저 측정하고 각 출력 주파수를 비교한다. 파형의 듀티비와 리셋 상태도 확인한다.

설계상 주의점

이번 출력은 관찰용 분주 신호다. 다른 레지스터의 클록으로 사용할 경우 클록 배선·타이밍 제약을 별도로 설계해야 한다.

4. 실험 18 · 스텝모터 위상 순서

설계 파일: stepper.v / 유효 출력: phase[3:0]

설계와 예상 결과

2비트 상태를 0, 1, 2, 3으로 순환시키고 조합논리로 위상을 출력한다. 상태 0부터 0011, 0110, 1100, 1001로 대응한다. 각 상태에서 두 출력이 활성화되며 네 번의 상승 에지 후 원래 위상으로 돌아온다.

사전 검증과 파형 해석

리셋 상태의 phase는 0이 아니라 3이다. 25, 35, 45, 55 ns에서 각각 6, C, 9, 3이 관측된다. 한 위상은 10 ns, 네 위상의 반복 주기는 40 ns다. 전체 실행에서 이 순서가 유지됨을 확인했다. 이는 구동 논리 검증이며 모터의 회전을 관측한 결과는 아니다.

```
example_project_hdl > src > @ stepper.v > {} lab5_stepper
1 timescale 1ns/1ps
2 module lab5_stepper(input wire clk, rst, output reg [3:0] phase);
3 // 원본의 2상 여자 순서. 실제 모터에는 별도의 구동 회로가 필요하다.
4 reg [1:0] state;
5 always @(posedge clk or posedge rst) begin
6     if (rst) state <= 0;
7     else state <= state + 1'b1;
8 end
9 always @* begin
10     case (state)
11         0: phase = 4'b0011;
12         1: phase = 4'b0110;
13         2: phase = 4'b1100;
14         3: phase = 4'b1001;
15         default: phase = 4'b0000;
16     endcase
17 end
18 endmodule
19
```

그림 5: 상태 레지스터와 4상 출력의 대응. 리셋은 상태를 0으로 설정한다.

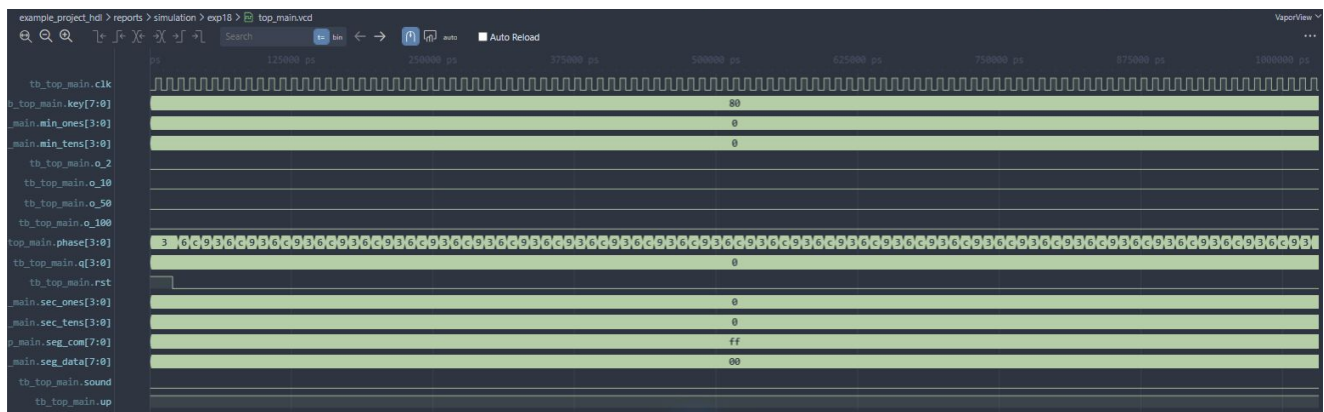


그림 6: phase의 16진수 표현 3 → 6 → C → 9 → 3. 비트 순서는 코드의 phase[3:0] 기준이다.

보드 검증 계획

모터 드라이버의 입력 순서·활성 레벨과 코일 연결을 확인한다. 모터 전류는 적절한 드라이버로 공급하고 위상 전환 주기를 실제 모터에 맞춘다.

추가 시험

실행 중 리셋에 따른 위상 복귀를 확인한다. 현재 회로에는 역회전·정지 입력이 없으므로 해당 기능을 구현했다고 해석하지 않는다.

5. 실험 19 · 피에조 단일 음 발생

설계 파일: tone.v / 유효 출력: sound

설계와 예상 결과

반주기 계수값 H 만큼 입력 에지를 세고 sound를 반전한다. 목표 294 Hz, 설정 클럭 1 MHz에서 정수 반올림으로 $H = 1701$ 이다. 따라서 실제 1 MHz 입력일 때 $f = 10^6 / (2H) \simeq 293.945$ Hz를 예상한다.

사전 검증과 파형 해석

이번 10 ns 클럭에서는 반주기가 17,010 ns, 주기가 34,020 ns다. sound는 17,025 ns에서 처음 1이 되고 34,035 ns에서 0이 된다. 그 뒤 전이 간격도 17,010 ns로 일치한다. key와 up의 변경은 이 실험의 음 출력에 영향을 주지 않는다.

```
example_project_hdl > src > @ tone.v > {} lab5_tone
1  `timescale 1ns/1ps
2  module lab5_tone(
3      input wire clk, rst, enable,
4      input wire [31:0] half_period,
5      output reg sound
6  );
7      // 출력 주파수 = 입력 클럭 / (2 * half_period)
8      reg [31:0] count;
9      always @(posedge clk or posedge rst) begin
10         if (rst) begin
11             count <= 0;
12             sound <= 0;
13         end else if (!enable || half_period == 0) begin
14             count <= 0;
15             sound <= 0;
16         end else if (count >= half_period - 1) begin
17             count <= 0;
18             sound <= ~sound;
19         end else count <= count + 1'b1;
20     end
21 endmodule
22
23 module lab5_piezo #(parameter integer CLK_HZ = 1000000)(
24     input wire clk, rst,
25     output wire sound
26 );
27     // 원본의 고정음 '레': 약 294Hz
28     localparam integer HALF_PERIOD = (CLK_HZ + 294) / (2 * 294);
29     lab5_tone tone(clk: clk, rst: rst, enable: 1'b1, half_period: HALF_PERIOD, sound: sound);
30 endmodule
31
```

그림 7: 공통 반주기 카운터와 294 Hz 설정을 사용하는 피에조 모듈.



그림 8: 전체 실행 구간의 sound. 시간축은 ps이며 17,010,000 ps마다 반전한다.

보드 검증 계획

실제 입력 클럭을 파라미터와 일치시킨 뒤 주기와 듀티비를 계측한다. 피에조의 종류와 구동 조건을 확인한 후 소리를 관찰한다.

추가 시험

공통 tone 모듈의 enable=0과 재활성화를 별도로 시험한다. 출력의 정수 분주 오차와 실측 클럭 오차를 구분하여 기록한다.

6. 실험 20 · 키 입력에 따른 피아노

설계 파일: piano.v, tone.v / 유효 신호: key[7:0], sound

설계와 예상 결과

key의 one-hot 입력을 8개 음의 반주기 값에 대응시킨다. 80은 262 Hz, 40은 294 Hz 설정이며 각각 $H = 1908, 1701$ 이다. 실제 1 MHz 입력이면 약 262.055, 293.945 Hz다. 입력이 없거나 여러 비트가 켜지면 default 분기로 무음 처리한다.

사전 검증과 파형 해석

이번 시나리오는 80 → 40 → 00이다. 초기 반주기는 19,080 ns이고 50,020 ns 이후 17,010 ns로 바뀐다. 키 변경 시 내부 count를 초기화하지 않으므로 새 키 입력 시각에서 반주기를 다시 재면 안 된다. 00 입력 후 첫 상승 에지인 100,025 ns에서 sound=0을 확인했다.

```
example_project_hdl > src > @ piano.v > ...
1  timescale 1ns/1ps
2  module lab5_piano #(parameter integer CLK_HZ = 1000000)(
3      input wire clk, rst,
4      input wire [7:0] key,
5      output wire sound
6  );
7
8      reg [31:0] half_period;
9      // 원본처럼 key[7]부터 도레미파솔라시도. 무입력/동시 입력은 무음.
10     always @* begin
11         case (key)
12             8'h80: half_period = (CLK_HZ + 262) / (2 * 262);
13             8'h40: half_period = (CLK_HZ + 294) / (2 * 294);
14             8'h20: half_period = (CLK_HZ + 330) / (2 * 330);
15             8'h10: half_period = (CLK_HZ + 349) / (2 * 349);
16             8'h08: half_period = (CLK_HZ + 392) / (2 * 392);
17             8'h04: half_period = (CLK_HZ + 440) / (2 * 440);
18             8'h02: half_period = (CLK_HZ + 494) / (2 * 494);
19             8'h01: half_period = (CLK_HZ + 523) / (2 * 523);
20             default: half_period = 0;
21         endcase
22     end
23     lab5_tone tone(clk: clk, rst: rst, enable: half_period != 0, half_period: half_period, sound: sound);
24 endmodule
```

그림 9: key[7]부터 낮은 도·레·미·파·솔·라·시·높은 도에 대응하는 정수 분주값.



그림 10: key의 80 → 40 전환 후 주기 감소와 00 입력 후 무음 상태.

추가 시뮬레이션

나머지 6개 음과 동시 키 입력은 이번 테스트벤치에 없었다. 각 키의 정상 주기, 잘못된 조합의 무음, 키 해제·재입력을 추가 확인한다.

보드 검증 계획

키의 실제 활성 레벨, 동기화와 점점 바운스를 확인한다. 음 전환 시 첫 펄스의 길이가 달라지는 현상과 정상상태의 주기를 구분해 측정한다.

7. 실험 21 · MM:SS 시계와 표시 스캔

설계 파일: watch.v / 유효 출력: 시간 4자리, seg_com, seg_data

설계와 예상 결과

1,000개 상승 에지마다 초를 증가시키고 00:00부터 59:59까지 계수한다. 초의 일의 자리 9에서 십의 자리로 올림하고, 초 59에서 분으로 올림한다. scan은 매 에지마다 다음 표시 자리를 선택한다. 네 자리 전체 스캔에는 4클럭이 필요하다.

사전 검증과 파형 해석

표시 초의 첫 증가는 10,015 ns이고 이후 간격은 10,000 ns다. 90,015 ns에 00:09, 100,015 ns에 00:10이 된다. 매 에지의 네 자리 값과 선택 신호·세그먼트 코드를 검사했다. 전체 구간 그림에서는 빠른 스캔 신호가 촘촘하게 뭉쳐 보인다.

```
example_project_hdl > src > @ watch.v > ...
1 | timescale 1ns/1ps
2 | module lab5_watch #(parameter integer CLK_HZ = 1000)(
3 |     input wire clk, rst,
4 |     output reg [3:0] min_tens, min_ones, sec_tens, sec_ones,
5 |     output reg [7:0] seg_data, seg_com
6 | );
7 |     reg [31:0] count;
8 |     reg [1:0] scan;
9 |     reg [3:0] digit;
10 |     // 한 클럭만 사용한다. CLK_HZ번마다 1초를 증가시킨다.
11 |     always @(posedge clk or posedge rst) begin
12 |         if (rst) begin
13 |             count <= 0; scan <= 0;
14 |             min_tens <= 0; min_ones <= 0; sec_tens <= 0; sec_ones <= 0;
15 |         end else begin
16 |             scan <= scan + 1'b1;
17 |             if (count == CLK_HZ - 1) begin
18 |                 count <= 0;
19 |                 if (sec_ones == 9) begin
20 |                     sec_ones <= 0;
21 |                     if (sec_tens == 5) begin
22 |                         sec_tens <= 0;
23 |                         if (min_ones == 9) begin
24 |                             min_ones <= 0;
25 |                             if (min_tens == 5) min_tens <= 0;
26 |                             else min_tens <= min_tens + 1'b1;
27 |                         end else min_ones <= min_ones + 1'b1;
28 |                     end else sec_tens <= sec_tens + 1'b1;
29 |                 end else sec_ones <= sec_ones + 1'b1;
30 |             end else count <= count + 1'b1;
31 |         end
32 |     end
end
```

그림 11: 시계의 계수와 자리올림을 담당하는 순차 블록 발췌. 표시 디코더는 같은 파일의 아래에 있다.



그림 12: sec_ones의 0-9 순환과 끝부분의 sec_tens=1. 종료 시 표시값은 00:10이다.

표시 인터페이스

자리 선택은 active-low이며 F7, FB, FD, FE 순서다. 세그먼트는 {a,b,c,d,e,f,g,dp} active-high이며 숫자 0은 FC, 1은 60이다. 리셋 때는 자리 FF, 데이터 00으로 표시를 끈다.

추가 시험·보드 검증

00:59 → 01:00 및 59:59 → 00:00은 별도 장시간 시험이 필요하다. 보드에서는 실제 1 kHz 기준과 자리·세그먼트 배선을 확인하고 깜박임과 잔상을 관찰한다.

8. 사전 검증 정리와 실험 수행 계획

시뮬레이션에서 확인한 사실과 다음 검증 항목

검증 결과 요약

카운터의 증가·감소와 순환, 분주비, 위상 순서, 선택한 두 음의 전이, 무음 전환, 시계의 00:10까지 계수 및 표시 코드가 예상과 일치했다. 각 실행의 종료 시각은 101,020 ns였고, 출력 비교는 리셋 해제 후 모든 상승 에지를 대상으로 했다. 스크린샷은 대표 구간이며 정량값은 저장된 전체 VCD에서 확인했다.

실험실 수행 순서

1. 사용할 보드의 클록·핀맵·I/O 전압과 주변장치의 활성 레벨을 확인한다.
2. 실험에 맞는 클록 조건 및 XDC 제약을 준비하고 합성·구현 결과를 확인한다.
3. 카운터와 분주기부터 보드에 구현하여 LED와 계측기로 기본 신호를 확인한다.
4. 드라이버를 통해 모터·피에조를 연결하고 위상·주기·키 입력에 따른 변화를 확인한다.
5. 시계의 시간 간격과 표시 자릿수를 확인하고 시뮬레이션의 예상값과 비교한다.
6. 관측 파형·사진·시연 영상과 차이가 발생한 조건을 실험 후 보고서에 기록한다.

예상되는 문제와 확인 방법

속도가 예상과 다르면 실제 클록 주파수와 파라미터를 먼저 비교한다. 카운터 LED나 표시 숫자가 뒤바뀌면 HDL 비트 순서와 핀 배치를 대조한다. 키 입력이 불안정하면 동기화·디바운스와 활성 레벨을 확인한다. 이 항목들은 예상 점검 절차이며 실제 발생·해결 사례로 기록한 것은 아니다.

실행 자료와 재현

보고서와 같은 폴더의 simulation/exp16~exp21에는 실행 소스, VCD, compile/elaborate/run 로그가 있다. simulation/manifest.json에는 파일별 SHA-256을 남겼다. 소스 캡처와 파형 캡처의 원본은 evidence에 보관하고, 본문에는 내용 변경 없이 여백을 잘라낸 인쇄용 이미지를 사용했다.

rerun.ps1에 Vivado bin 경로를 전달하면 별도 임시 폴더에서 다시 실행하고 VCD를 새로 생성할 수 있다. 보고서에 사용한 원본 실행 자료는 보존한다. 검증 스크립트는 저장된 VCD로 결과 JSON을 다시 만든다.

보고서 제출본에 적용할 때

예시의 이름·학번과 실행 결과를 그대로 제출하지 않고 본인의 실험 내용을 작성한다. 표지의 로컬 소스 정보는 본인의 GitHub 저장소 주소·제출 태그·커밋으로 바꾼다. 본문에는 핵심 코드와 설명을 남기고 전체 소스는 해당 버전에서 확인 가능하게 한다. 보드 실험 전이므로 실제 동작 사진이나 계측값을 임의로 넣지 않는다.

참고문헌

- [1] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed., Pearson, 2013. 순차논리 및 레지스터·카운터 관련 장. 출판 정보: [Pearson Computer Science Catalogue \(2017\)](#).